

JavaScript Array CRUD – Beginner's Guide

Objective

The purpose of this program is to understand how CRUD (Create, Read, Update, Delete) operations work using a JavaScript array.

Instead of storing data in a database, this program stores student names inside an array.

What is CRUD?

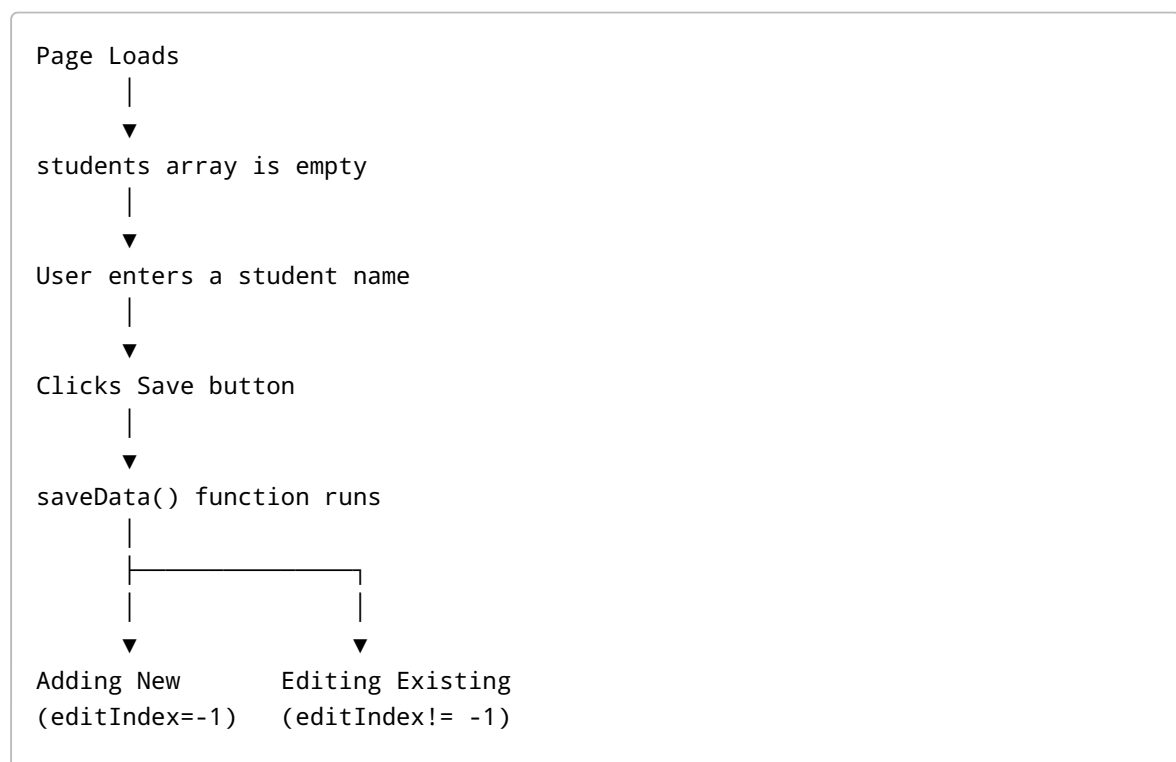
CRUD stands for:

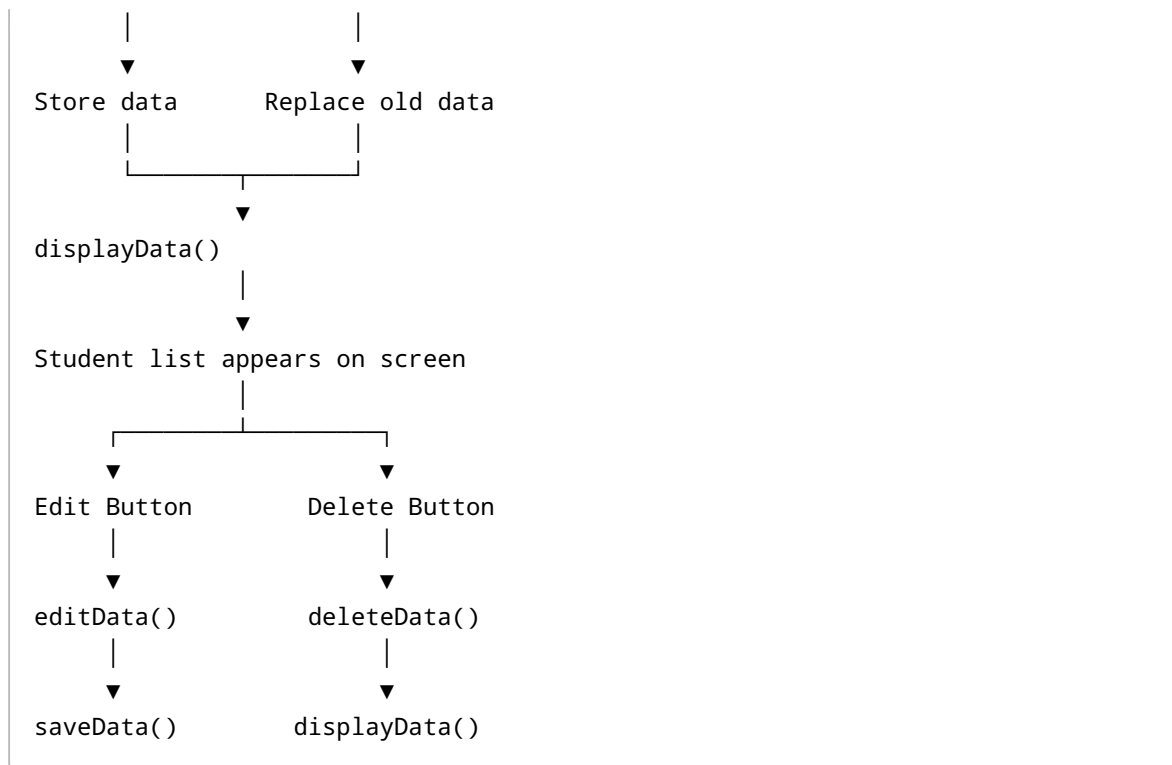
- **C - Create** → Add new data
- **R - Read** → Display existing data
- **U - Update** → Edit existing data
- **D - Delete** → Remove existing data

This program performs all four operations on a JavaScript array.

Overall Program Flow

The program follows the flow shown below.





Variables Used

students

```
let students = [];
```

This array stores all student names.

Initially,

```
students = []
```

After adding students,

```
students = ["Rahul", "Amit", "Priya"]
```

Everything in the program depends on this array.

editIndex

```
let editIndex = -1;
```

This variable tells the program whether the user is:

- Adding a new student
- Editing an existing student

When

```
editIndex = -1
```

it means

Add a new student.

When

```
editIndex = 2
```

it means

Edit the student stored at index 2.

Function 1 : saveData()

```
function saveData()
```

This is the most important function of the program.

It runs whenever the **Save** button is clicked.

Its job is to

- Read the textbox
 - Validate input
 - Add a new student OR update an existing one
 - Refresh the list
-

Step 1 : Read textbox value

```
let name = document.getElementById("name").value;
```

This gets whatever the user typed.

Example

Textbox

Rahul

Now

```
name = "Rahul"
```

Step 2 : Validation

```
if(name=="")
```

Checks whether the textbox is empty.

If empty,

```
alert("Enter a name");
```

shows a popup.

Then

```
return;
```

stops the function immediately.

Without this check, empty names could be added.

Step 3 : Decide Add or Edit

The program checks

```
if(editIndex===-1)
```

Case 1 : Add

If editIndex is -1,

```
students.push(name);
```

adds the new name to the array.

Example

Before

```
["Rahul","Amit"]
```

After

```
["Rahul","Amit","Priya"]
```

Case 2 : Update

Otherwise,

```
students[editIndex]=name;
```

replaces the old value.

Example

Before

```
["Rahul","Amit","Priya"]
```

Suppose

```
editIndex = 1
```

New name

Akash

Result

["Rahul", "Akash", "Priya"]

The old value is replaced.

Step 4 : Reset edit mode

```
editIndex = -1;
```

After editing, the program returns to **Add Mode**.

Otherwise every Save click would continue editing the same student.

Step 5 : Clear textbox

```
document.getElementById("name").value="";
```

The textbox becomes empty.

Before

Rahul

After

(blank)

Step 6 : Display updated list

```
displayData();
```

This redraws the student list.

Function 2 : displayData()

```
function displayData()
```

Purpose

Display every student stored in the array.

Whenever

- Save
- Edit
- Delete

is performed, this function is called.

Step 1 : Get UL

```
let list=document.getElementById("list");
```

Gets

```
<ul id="list">
```

Step 2 : Clear previous data

```
list.innerHTML="";
```

Why?

Suppose this line is removed.

Old data

```
Rahul  
Amit
```

After adding Priya

```
Rahul  
Amit  
Rahul  
Amit  
Priya
```

Everything gets repeated.

So we first remove the old list and rebuild it.

Step 3 : Loop through array

```
for(i=0;i<students.length;i++)
```

The loop runs once for every student.

Suppose

```
students=["Rahul","Amit","Priya"]
```

Iterations

```
i=0  
Rahul  
  
i=1  
Amit  
  
i=2  
Priya
```

Step 4 : Create HTML

Each iteration creates

```
<li>  
Student Name  
Edit Button  
Delete Button  
</li>
```

using template literals.


```
list.innerHTML += `  
<li>  
  ${students[i]}  
  <button onclick="editData(${i})">Edit</button>  
  <button onclick="deleteData(${i})">Delete</button>  
</li>  
`;  
`;
```

`${students[i]}` inserts the current student's name into the HTML.

Example

students[1]

becomes

Amit

Function 3 : editData()

```
function editData(index)
```

Runs when the Edit button is clicked.

Suppose

Rahul
Amit
Priya

User clicks

Edit

for Amit.

The function receives

index = 1

Step 1 : Put data into textbox

```
document.getElementById("name").value=students[index];
```

Textbox becomes

```
Amit
```

Now the user can change it.

Step 2 : Store index

```
editIndex=index;
```

Now

```
editIndex=1
```

Next time Save is clicked,

the program knows

Update student at index 1

instead of adding a new one.

Function 4 : deleteData()

```
function deleteData(index)
```

Runs when Delete button is clicked.

Suppose

```
students=["Rahul","Amit","Priya"]
```

Delete

```
Amit
```

The function receives

```
index=1
```

Remove from array

```
students.splice(index,1);
```

Meaning

- Start from index
- Remove 1 element

Before

```
["Rahul","Amit","Priya"]
```

After

```
["Rahul","Priya"]
```

Refresh list

```
displayData();
```

The screen updates immediately.

Why displayData() is called again and again?

Whenever data changes,

the screen must also change.

Therefore

```
Add
  ↓
displayData()

Edit
  ↓
displayData()

Delete
  ↓
displayData()
```

This keeps the UI synchronized with the array.

Example Execution

Initially

```
students=[]
```

User enters

```
Rahul
```

After Save

```
students=["Rahul"]
```

Screen

```
Rahul
```

User enters

```
Amit
```

After Save

```
students=["Rahul","Amit"]
```

Screen

```
Rahul  
Amit
```

User clicks Edit on Rahul.

Textbox

```
Rahul
```

User changes it to

```
Rohan
```

After Save

```
students=["Rohan","Amit"]
```

Screen

```
Rohan  
Amit
```

User clicks Delete on Amit.

Array

```
["Rohan"]
```

Screen

```
Rohan
```

Summary

Function	Purpose
<code>saveData()</code>	Adds a new student or updates an existing one.
<code>displayData()</code>	Displays all students from the array on the webpage.
<code>editData(index)</code>	Loads the selected student's name into the textbox and enables edit mode.
<code>deleteData(index)</code>	Removes the selected student from the array and refreshes the list.

Key JavaScript Concepts Learned

- Arrays (`push()` , `splice()`)
- Variables
- Functions
- `if...else` statements
- `for` loops
- DOM Manipulation (`getElementById()`)
- `innerHTML`
- Event Handling (`onclick`)
- Template Literals (``${}``)
- CRUD Operations
- Input Validation
- Dynamic HTML Generation

By understanding this simple project, you build the foundation for implementing CRUD operations with databases, APIs, React, Node.js, and other modern web technologies in future projects.